

Midterm Examination, Spring-2024

Course Title: Data Structures

Course Code: CSE-2321

(This PDF contains ChatGPT generated answers. While it aims to provide useful insights, some content may be inaccurate or imprecise. Use this as a study guide and confirm facts when needed.)

1. a) A professor keeps a class list containing the following data for each student:
Name, ID Number, Section, Total Marks, Final Grade

- i) State the entities, attributes, and entity set of the list.
- ii) Which attribute can serve as the primary key for the list?

Ans: i) **Entities, Attributes, and Entity Set:**

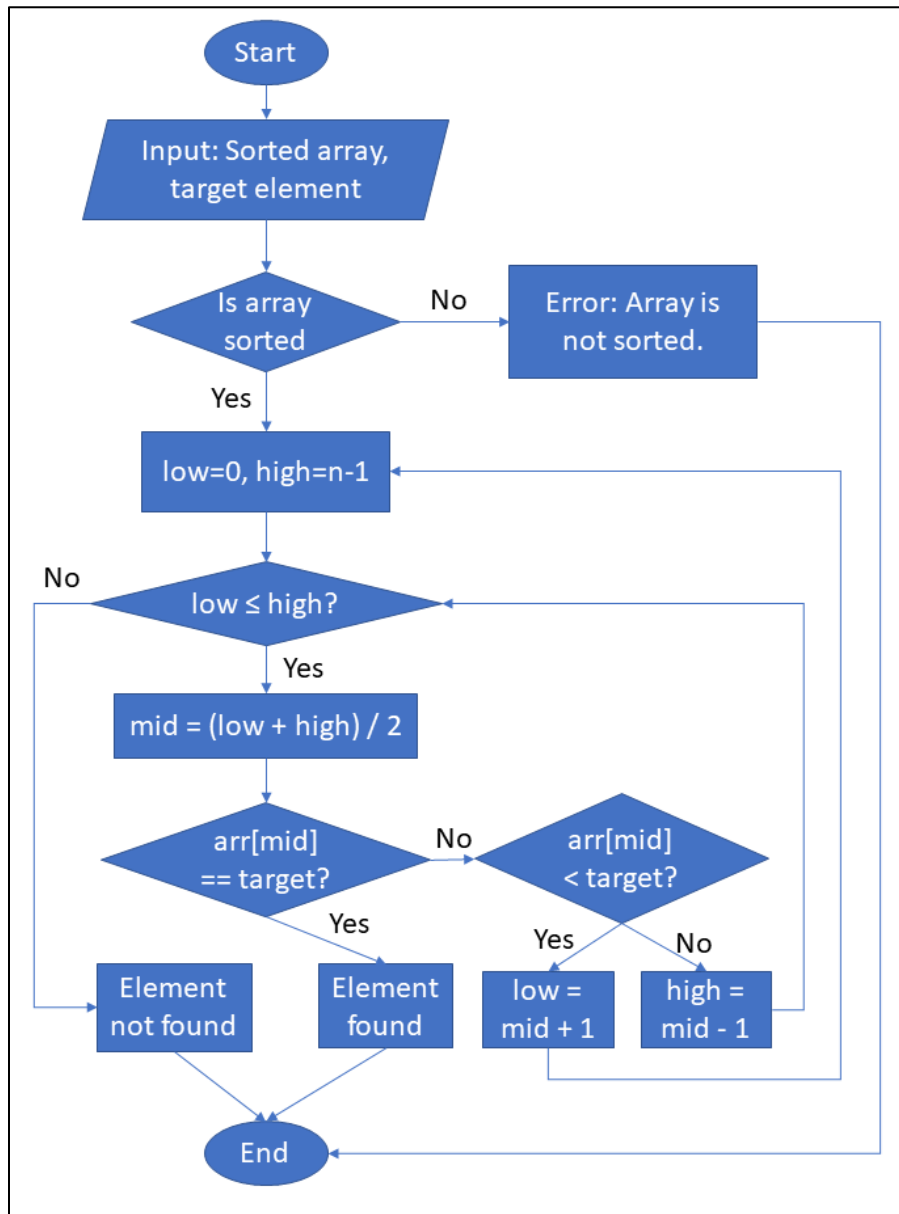
- **Entities:** Students
- **Attributes:** Name, ID Number, Section, Total Marks, Final Grade
- **Entity Set:** Class List (contains all students and their details)

ii) **Primary Key:**

- **ID Number** can serve as the primary key because it uniquely identifies each student in the list.

b) Draw a flowchart for the binary search algorithm

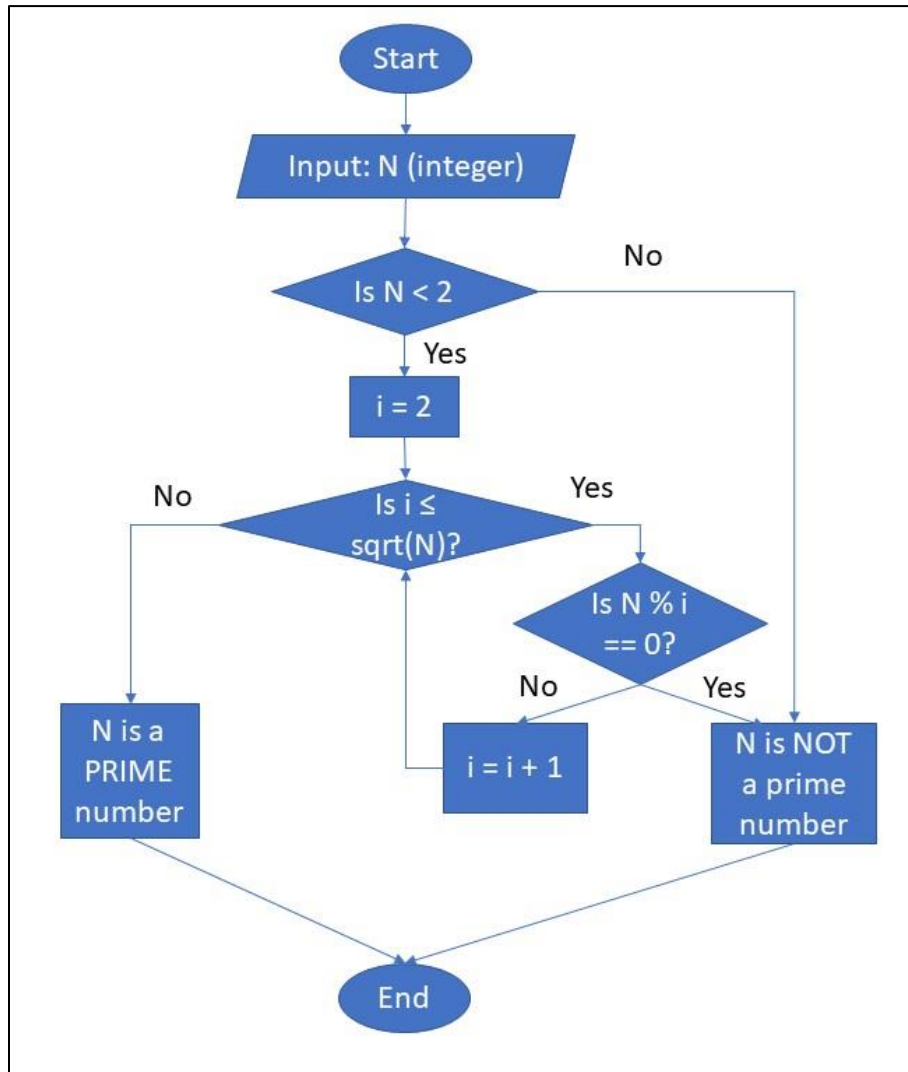
Ans:



Or

Draw a **flowchart** to determine whether a given number **N** is **prime** or not.

Ans:



c) What do you mean by **space complexity**? What is the **space complexity** of the following functions? Explain.

```
(i)
void swap()
{
    int a, b, temp;
    temp = a;
    a = b;
    b = temp;
}
```

```
(ii)
int sum(int a[], int n)
{
    int i, sum = 0;
    for(i = 0; i < n; i += 2)
        sum += a[i];
    return sum;
}
```

Ans: Space complexity refers to the **amount of memory space** required by an algorithm **relative to the input size**. It includes:

1. **Fixed part** (constants, program variables).
2. **Variable part** (dynamic memory allocations, function calls, recursion stack, etc.).

(i) • This function uses **3 integer variables (a, b, temp)**.

- Each integer takes **O(1)** space.
- Since it does not depend on the input size, the **space complexity is O(1)**.

(ii) • Uses **one integer array a [] of size n** (already passed as an argument).

- Additional **integer variables: i, sum** → **O(1) space**.
- No extra space is allocated apart from the input array.
- **Space complexity = O(1)** (since array a [] is not created inside the function).

d) For the following pattern **P** and text **T**, find the number **C** of comparisons to find the **INDEX** of **P** in **T** using the **pattern matching algorithm** you studied. You have to show each step.

$P = \text{ccd}$, $T = \text{cbcdcdcdcdcdcdcd}$

Ans: Using Brute Force Pattern Matching:

We compare **P** with every substring of **T** of length 3.

C	Substring	Comparison with "ccd"	Match?
1	"cbc"	No	✗
2	"bcd"	No	✗
3	"cdc"	No	✗
4	"dcd"	No	✗
5	"cdd"	No	✗
6	"ddd"	No	✗
7	"ddc"	No	✗
8	"dcd"	No	✗
9	"cdd"	No	✗
10	"ddd"	No	✗
11	"ddc"	No	✗
12	"dcd"	No	✗
13	"cdd"	No	✗
14	"ddc"	No	✗
15	"dcc"	No	✗
16	"ccd"	Yes	☑

Number of comparisons (C) = 16

2.

a) Write an algorithm to **insert** an element **ITEM** to the **Kth** position of a linear array **LA**. What is the **time complexity** of your algorithm?

Ans: **Insert an Element ITEM at Kth Position in a Linear Array LA**

```
INSERT (LA, N, ITEM, K)
```

Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm inserts an element ITEM into Kth position in LA.

```
1. Set J:=N
2. Repeat Steps 3 and 4 while J ≥ K
3. Set LA[J+1]:=LA[J]
4. Set J:=J-1
5. Set LA[K]:=ITEM
6. Set N:=N+1
7. Exit
```

Time Complexity Analysis:

- **Best case (Insertion at the end)** $\rightarrow O(1)$
- **Worst case (Insertion at the start)** $\rightarrow O(N)$ (shifting N elements)
- **Average case** $\rightarrow O(N/2) \approx O(N)$

Final Complexity: $O(N)$ (due to shifting elements).

b) Consider an array B(1:8, -5:5, -10:5):

i) Find the length of each dimension and the number of elements in B.

ii) Suppose Base(B) = 150 and W = 4 words per memory cell for B.

Find the effective indices E_1, E_2, E_3 , and the address of the element B[2, -3, 2] assuming B is stored in row-major order.

Ans: i) The array **B(1:8, -5:5, -10:5)** is a **3D array** with the given index ranges for each dimension:

- First dimension (1 to 8):

$$\text{Length} = 8 - 1 + 1 = 8$$

- Second dimension (-5 to 5):

$$\text{Length} = 5 - (-5) + 1 = 11$$

- Third dimension (-10 to 5):

$$\text{Length} = 5 - (-10) + 1 = 16$$

Total number of elements in B:

$$8 \times 11 \times 16 = 1408$$

ii) Step 1: Effective Indices

To calculate the effective indices **E_1, E_2 , and E_3** , we use the formula:

$$E_i = \text{Actual index} - \text{Lower bound} + 1$$

For **B[2, -3, 2]**:

- E_1 (for $i = 2$, first dimension **1 to 8**):

$$E_1 = 2 - 1 + 1 = 2$$

- E_2 (for $j = -3$, second dimension **-5 to 5**):

$$E_2 = (-3) - (-5) + 1 = 3$$

- E_3 (for $k = 2$, third dimension **-10 to 5**):

$$E_3 = 2 - (-10) + 1 = 13$$

Thus, **$(E_1, E_2, E_3) = (2, 3, 13)$** .

Step 2: Address Calculation

Since **B** is stored in row-major order, the address of an element in a **3D array** is calculated as:

$$\text{Address}(B[i, j, k]) = \text{Base} + [(E_1 - 1) \times S_2 \times S_3] + [(E_2 - 1) \times S_3] + [(E_3 - 1)] \times W$$

Where:

- **Base(B) = 150**
- **W = 4** (words per memory cell)
- **S₃ = 16** (size of the third dimension)
- **S₂ = 11 × 16 = 176** (size of the second and third dimension)

Substituting the values:

$$\begin{aligned}\text{Address}(B[2, -3, 2]) &= 150 + [(2 - 1) \times 176] + [(3 - 1) \times 16] + [(13 - 1)] \times 4 \\ &= 150 + (1 \times 176) + (2 \times 16) + (12 \times 4) \\ &= 150 + 176 + 32 + 48 \\ &= 406\end{aligned}$$

c) Given a string **STRUCTURES**, find the **number of comparisons (C)** and **number of interchanges (D)** needed to **sort** the string **alphabetically** using the **bubble sort algorithm**. Show each step.

Ans:

Sorting Process

Pass 1

Compare adjacent elements and swap if needed.

Index	Before	Swap?	After
0-1	STRUCTURES	✗	STRUCTURES
1-2	TR	☑	SRTUSTRUCTURES
2-3	TU	✗	SRTUSTRUCTURES
3-4	UC	☑	SRTCUTURES
4-5	UT	☑	SRTCTUURES
5-6	UU	✗	SRTCTUURES
6-7	UR	☑	SRTCTURUES
7-8	UE	☑	SRTCTUREUS
8-9	US	☑	SRTCTURESU

Comparisons in Pass 1: 9

Swaps in Pass 1: 5

Last element (U) is now sorted.

Pass 2

Index	Before	Swap?	After
0-1	SR	☑	RSTCTURESU
1-2	ST	✗	RSTCTURESU
2-3	TC	☑	RSCTTURESU
3-4	TT	✗	RSCTTURESU
4-5	TU	✗	RSCTTURESU
5-6	UR	☑	RSCTTRUESU
6-7	UE	☑	RSCTTREUSU
7-8	US	☑	RSCTTRESUU

Comparisons in Pass 2: 8

Swaps in Pass 2: 4

Second last element (U) is now sorted.

Pass 3

Index	Before	Swap?	After
0-1	R S	✗	R S C T T R E S U U
1-2	S C	☑	R C S T T R E S U U
2-3	S T	✗	R C S T T R E S U U
3-4	T T	✗	R C S T T R E S U U
4-5	T R	☑	R C S T R T E S U U
5-6	T E	☑	R C S T R E T S U U
6-7	T S	☑	R C S T R E S T U U

Comparisons in Pass 3: 7

Swaps in Pass 3: 3

Pass 4

Index	Before	Swap?	After
0-1	R C	☑	C R S T R E S T U U
1-2	R S	✗	C R S T R E S T U U
2-3	S T	✗	C R S T R E S T U U
3-4	T R	☑	C R S R T E S T U U
4-5	T E	☑	C R S R E T S T U U
5-6	T S	☑	C R S R E S T T U U

Comparisons in Pass 4: 6

Swaps in Pass 4: 3

Pass 5

Index	Before	Swap?	After
0-1	C R	✗	C R S R E S T T U U
1-2	R S	✗	C R S R E S T T U U
2-3	S R	☑	C R R S E S T T U U
3-4	S E	☑	C R R E S S T T U U
4-5	S S	✗	C R R E S S T T U U

Comparisons in Pass 5: 5

Swaps in Pass 5: 2

Pass 6

Index	Before	Swap?	After
0-1	C R	✗	CRRESSTTUU
1-2	R R	✗	CRRESSTTUU
2-3	R E	☑	CRERSSTTUU
3-4	R S	✗	CRERSSTTUU

Comparisons in Pass 6: 4

Swaps in Pass 6: 1

Pass 7

Index	Before	Swap?	After
0-1	C R	✗	CRERSSTTUU
1-2	R E	☑	CERRSSTTUU

Comparisons in Pass 7: 3

Swaps in Pass 7: 1

Pass 8

Index	Before	Swap?	After
0-1	C E	✗	CERRSSTTUU

Comparisons in Pass 8: 2

Swaps in Pass 8: 0

Since no swaps were made in **Pass 8**, the array is fully sorted.

Final Sorted String:

C E R R S S T T U U

Final Count

Total Comparisons (C)	Total Swaps (D)
$9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 = 44$	$5 + 4 + 3 + 3 + 2 + 1 + 1 + 0 = 19$

Thus, **Number of Comparisons (C) = 44**

Number of Swaps (D) = 19

Or

Modify the **bubble sort** algorithm so that it will stop early if it detects that the list is already sorted.
Write the **modified bubble sort algorithm**.

Ans:

```
BUBBLE_SORT (DATA, N)
Here DATA is an array with N elements. This algorithm sorts the elements in DATA.
1. Repeat Steps 2, 3, and 4 for K = 1 to N-1
2. Set PTR := 1
3. Set SWAPPED := False → (New step: track if swaps occur)
4. Repeat while PTR <= N-K:
    (a) If DATA[PTR] > DATA[PTR+1], then:
        - Interchange DATA[PTR] and DATA[PTR+1]
        - Set SWAPPED := True
    (b) Set PTR := PTR + 1
5. If SWAPPED = False, Exit → (New step: stop early if no swaps)
6. Exit
```

d) Write down the **worst case & average case complexity** for the following algorithms:

- i) Binary search
- ii) Linear search
- iii) Bubble sort

Ans:

(i) *Binary Search*

- **Worst Case Complexity: $O(\log N)$** (when the element is at the last level of the search or not found)
- **Average Case Complexity: $O(\log N)$** (since each search step halves the search space)

(ii) *Linear Search*

- **Worst Case Complexity: $O(N)$** (when the element is at the last position or not present)
- **Average Case Complexity: $O(N)$** (since on average, we search half the elements)

(iii) *Bubble Sort*

- **Worst Case Complexity: $O(N^2)$** (when the array is sorted in reverse order)
- **Average Case Complexity: $O(N^2)$** (since swaps occur in most cases, leading to N^2 operations)

3. a)

What is a **stack**? Write a procedure to **PUSH** an item onto a stack and **POP** an item from the stack.

Ans: A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle. It means the last element added is the first to be removed.

- **PUSH** operation: Adds an item to the top of the stack.
- **POP** operation: Removes an item from the top of the stack.

Procedures for PUSH and POP (Pseudocode):

```
Procedure PUSH(STACK, ITEM, TOP, N)
1. If TOP = N, then print "Stack Overflow" and return.
2. Set TOP := TOP + 1
3. STACK[TOP] := ITEM
4. Return

Procedure POP(STACK, ITEM, TOP)
1. If TOP = 0, then print "Stack Underflow" and return.
2. ITEM := STACK[TOP]
3. Set TOP := TOP - 1
4. Return ITEM
```

b)

Suppose the following stack of integers is in memory where **STACK** is allocated **N = 8** memory cells:

TOP = 4
STACK: 11, 22, 33, 44, __, __, __, __

Describe the **STACK** and **TOP** as the following operations take place –

- POP(STACK, ITEM)**
- PUSH(STACK, 77)**
- PUSH(STACK, 99)**
- POP(STACK, ITEM)**

Ans:

i) POP(STACK, ITEM)

- Removes 44 (topmost element).
- **New Stack:** [11, 22, 33, __, __, __, __, __]
- **New TOP:** 3

ii) PUSH(STACK, 77)

- Inserts 77 at TOP+1.
- **New Stack:** [11, 22, 33, 77, __, __, __, __]
- **New TOP:** 4

iii) PUSH(STACK, 99)

- Inserts 99 at TOP+1.
- **New Stack:** [11, 22, 33, 77, 99, __, __, __]
- **New TOP:** 5

iv) POP(STACK, ITEM)

- Removes 99.
- **New Stack:** [11, 22, 33, 77, __, __, __, __]
- **New TOP:** 4

c)

Consider the following **postfix expression**:

P: 3, 2, +, 2, ^, 7, 4, -, 2, *, +, 5, -

Evaluate **P**, using the algorithm you have studied.

Ans: **Postfix Expression:**

P: 3, 2, +, 2, ^, 7, 4, -, 2, *, +, 5, -

Algorithm:

1. Read elements from left to right.
2. Push numbers onto the stack.
3. When an operator appears, pop operands, apply the operation, and push the result back.

Step-by-step evaluation:

Step	Stack After Step	Operation
Push 3	[3]	-
Push 2	[3, 2]	-
+	[5]	$(3 + 2)$
Push 2	[5, 2]	-
^	[25]	$(5^2 = 25)$
Push 7	[25, 7]	-
Push 4	[25, 7, 4]	-
-	[25, 3]	$(7 - 4 = 3)$
Push 2	[25, 3, 2]	-
*	[25, 6]	$(3 * 2 = 6)$
+	[31]	$(25 + 6 = 31)$
Push 5	[31, 5]	-
-	[26]	$(31 - 5 = 26)$

Final Answer: 26

d) Consider the following infix expression Q:

Q: $((A + B) / D) ^ ((E - F) * G)$

- **Translate Q into its equivalent postfix expression P** using the algorithm you studied.

Ans: Conversion Steps (Using Stack-Based Algorithm):

1. **Operators with higher precedence are placed later.**
2. **Parentheses dictate evaluation order.**
3. **Order of precedence:**
 - ^ (Exponentiation) – Highest
 - * / (Multiplication & Division) – Medium
 - + - (Addition & Subtraction) – Lowest

Conversion to Postfix:

Q: $((A + B) / D) ^ ((E - F) * G)$

P: $A B + D / E F - G * ^$

☒ **Postfix Expression:** $A B + D / E F - G * ^$

Or

- Write an algorithm that receives a **bracket sequence** and tells whether the sequence is **correct** or **incorrect**.

For example, “()”, “()[]{}”, “({})[]” are correct sequence while, “({)[]”, “{ }” are not.

[You can assume that the required **data structure** and its **operational algorithms** are available to solve this problem. You don't need to re-implement them.]

Ans:

```
Algorithm CHECK_BALANCED(BRACKETS)
1. Initialize an empty stack.
2. For each character in BRACKETS:
  a) If it's an opening bracket ( ( { [ ), push it onto the stack.
  b) If it's a closing bracket ( ) } ] ):
    - If stack is empty, return "INVALID".
    - Pop the stack and check if it matches the corresponding opening bracket.
    - If mismatched, return "INVALID".
3. If stack is empty after processing, return "VALID".
4. Else, return "INVALID".
```